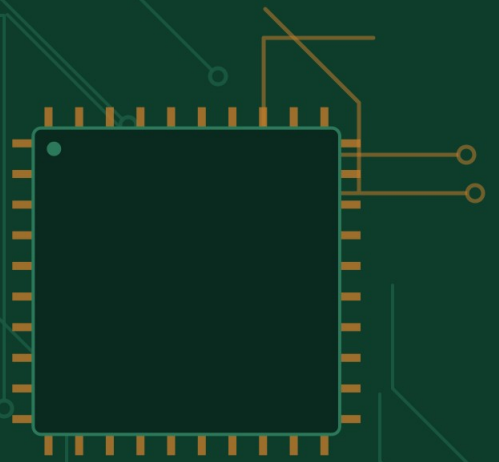
PERTEMUAN 13

IoT: Wi-Fi dan MQTT

Dari sensor di meja sampai pesan di peramban



Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

Tujuan Pembelajaran

- Menjelaskan arsitektur **IoT**: perangkat → jaringan → broker → aplikasi
- Memindai Wi-Fi (SSID, RSSI, kanal) dan menyambung dalam mode **STA**
- Menjelaskan **MQTT**: topik, wildcard, QoS, retained, LWT, keep alive
- Mengirim data sensor **JSON** ke broker dan memantaunya di MQTTX
- Mengendalikan LED lewat MQTT dengan sambung ulang **tanpa memblokir**

Peta Materi Hari Ini

01 Arsitektur IoT

02 Wi-Fi ESP32: STA/AP, RSSI, DHCP

03 Wokwi vs board asli + menyimpan sandi

04 MQTT: publish/subscribe, topik, QoS

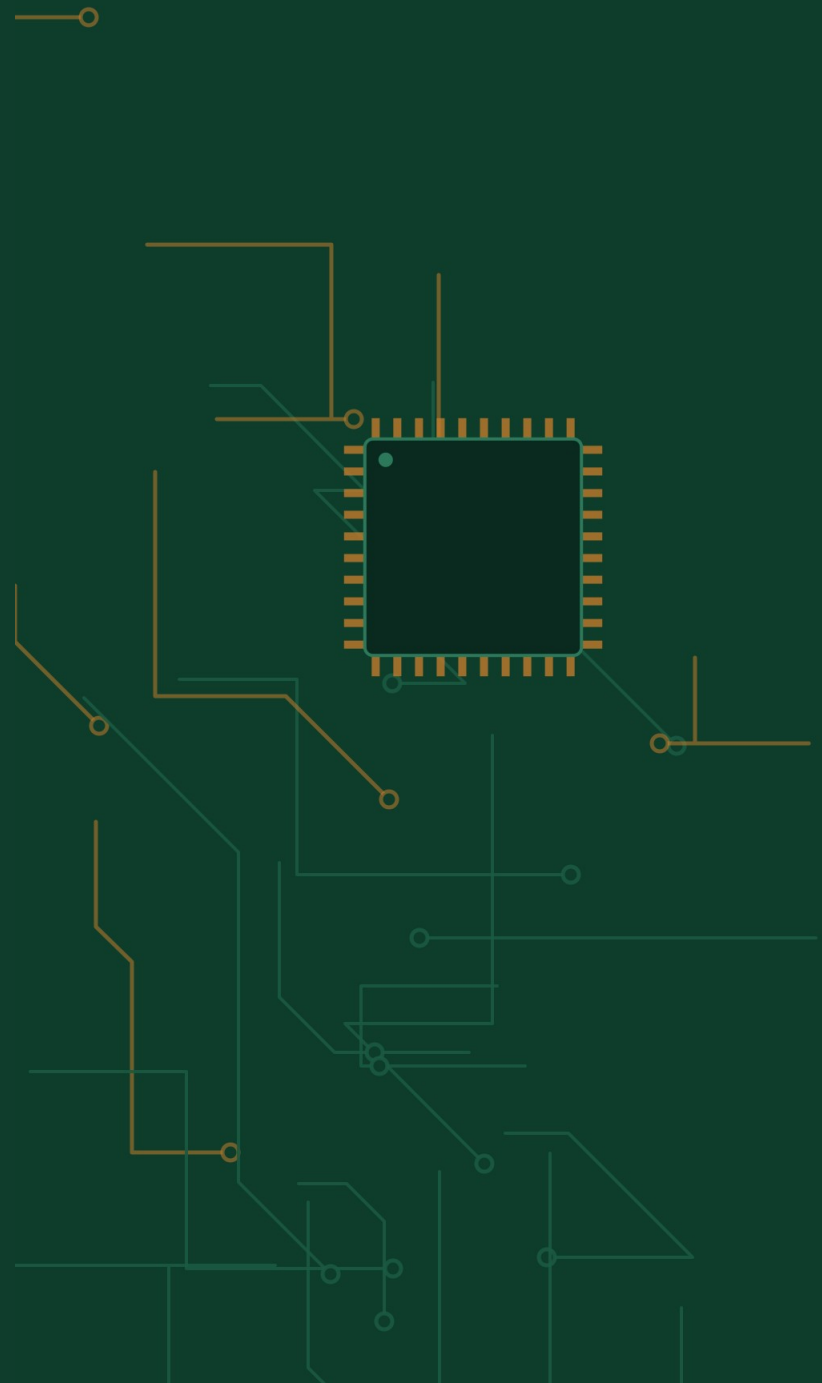
05 Retained, LWT, keep alive, ID klien

06 Praktikum: pindai Wi-Fi, kirim data, kendali LED

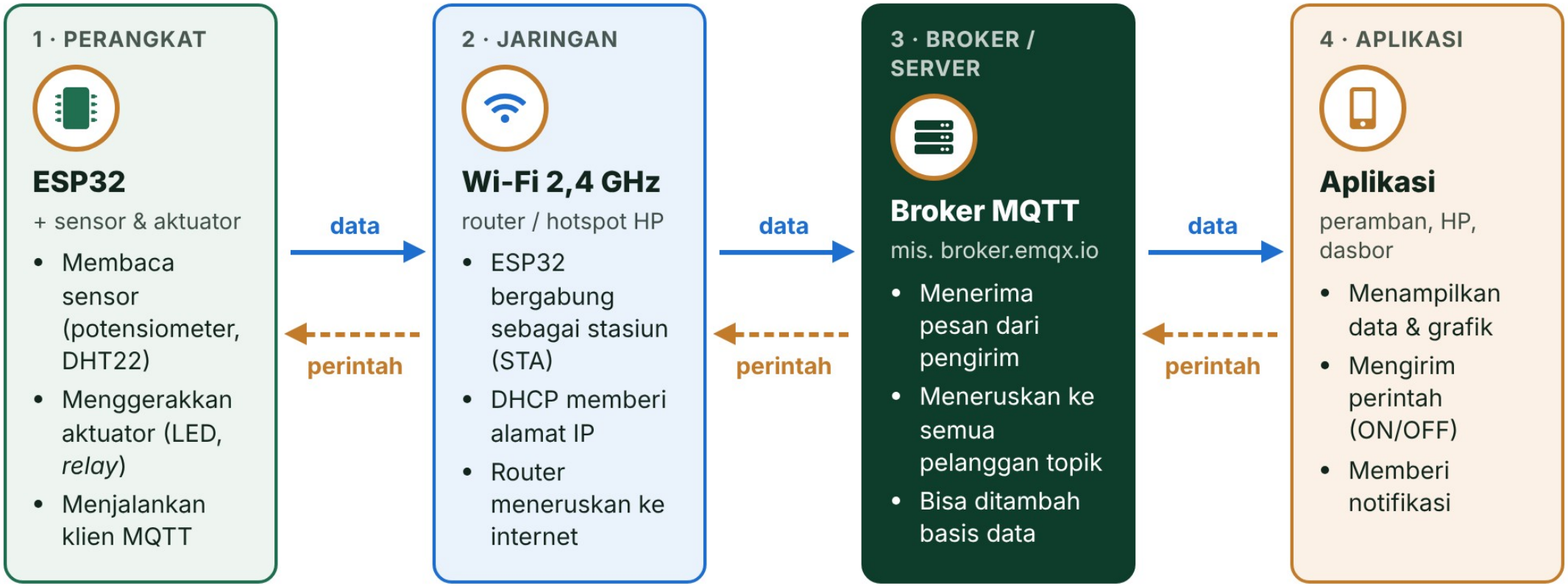
01

IoT dan Wi-Fi ESP32

Membawa data dari meja kerja ke internet



Arsitektur IoT: Empat Lapis

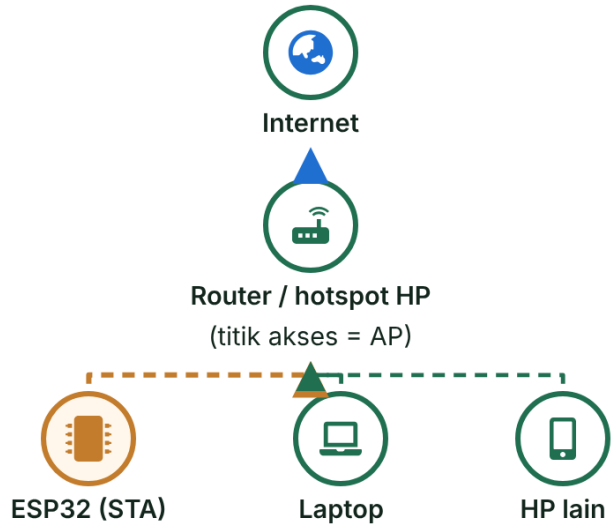


Di pertemuan ini: potensiometer → ESP32 → Wi-Fi (Wokwi-GUEST/hotspot) → **broker.emqx.io** → MQTTX Web di peramban.

Mode STA dan Mode AP

Mode STA (stasiun)

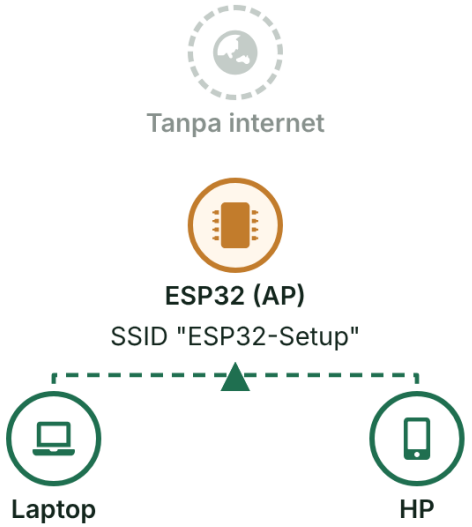
ESP32 menjadi **tamu** di jaringan yang sudah ada.




```
WiFi.mode(WIFI_STA);
WiFi.begin("NamaHotspot", "sandi");
```

Mode AP (titik akses)

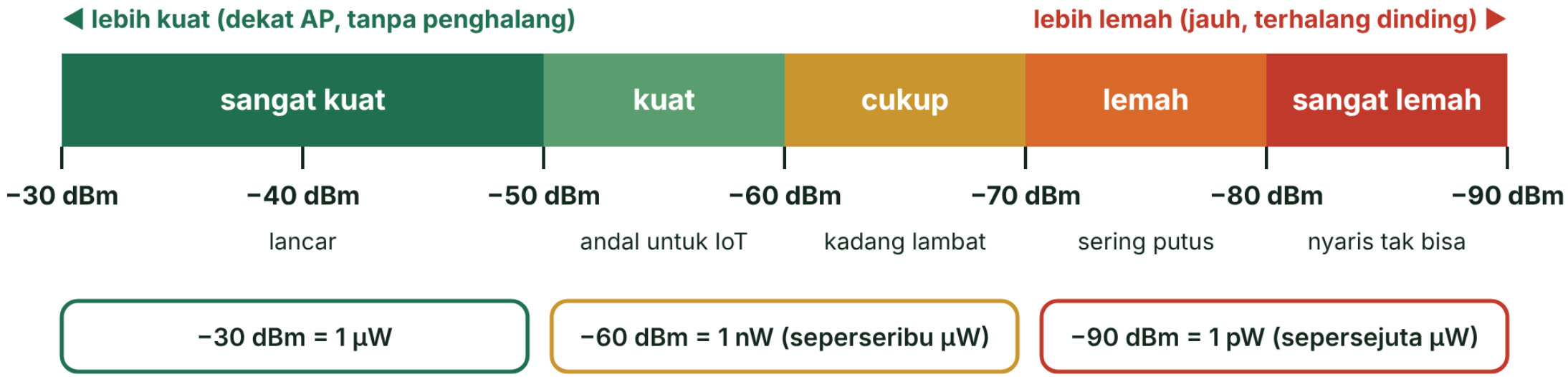
ESP32 menjadi **tuan rumah** jaringannya sendiri.




```
WiFi.mode(WIFI_AP);
WiFi.softAP("ESP32-Setup", "sandi123");
```

Garis putus-putus = sambungan nirkabel. Ada juga mode gabungan `WIFI_AP_STA`: tamu di router sekaligus tuan rumah bagi HP.

RSSI: Makin Mendekati 0, Makin Kuat



Pembagian kategori ini patokan kasar, bukan standar resmi. Setiap turun 10 dB, daya tinggal 1/10; turun 3 dB, daya tinggal ±separuhnya.

dBm itu logaritmis: −30 dBm = 1 µW, −60 dBm = 1 nW. Turun 10 dB = daya tinggal 1/10.

Menyambung ke Wi-Fi

```
contoh singkat

#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);              // jadi "tamu"
  WiFi.begin("Wokwi-GUEST", "", 6); // SSID, sandi, kanal
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(WiFi.localIP());   // 10.10.0.2
}
```

- `WiFi.scanNetworks()`: pindai, hasilkan jumlah jaringan
- `WiFi.status()` = `WL_CONNECTED` (3) bila tersambung
- `WiFi.localIP()`: alamat dari **DHCP**
- Kanal 6 di Wokwi: sambung ±4 detik lebih cepat

Wi-Fi di Wokwi vs Board Asli

Wokwi

- SSID `Wokwi-GUEST`, tanpa sandi, kanal 6
- IP selalu `10.10.0.2`
- Hanya sambungan keluar; lalu lintas dipantau
- Tidak perlu berkas `rahasia.h`

Board Asli

- *Hotspot* HP pita **2,4 GHz**
- IP diberikan DHCP *hotspot*
- Wi-Fi kampus (login/WPA2-Enterprise) sering gagal
- Buat `include/rahasia.h` dari contoh

Sandi Wi-Fi Jangan Ikut di-Commit

src/main.cpp

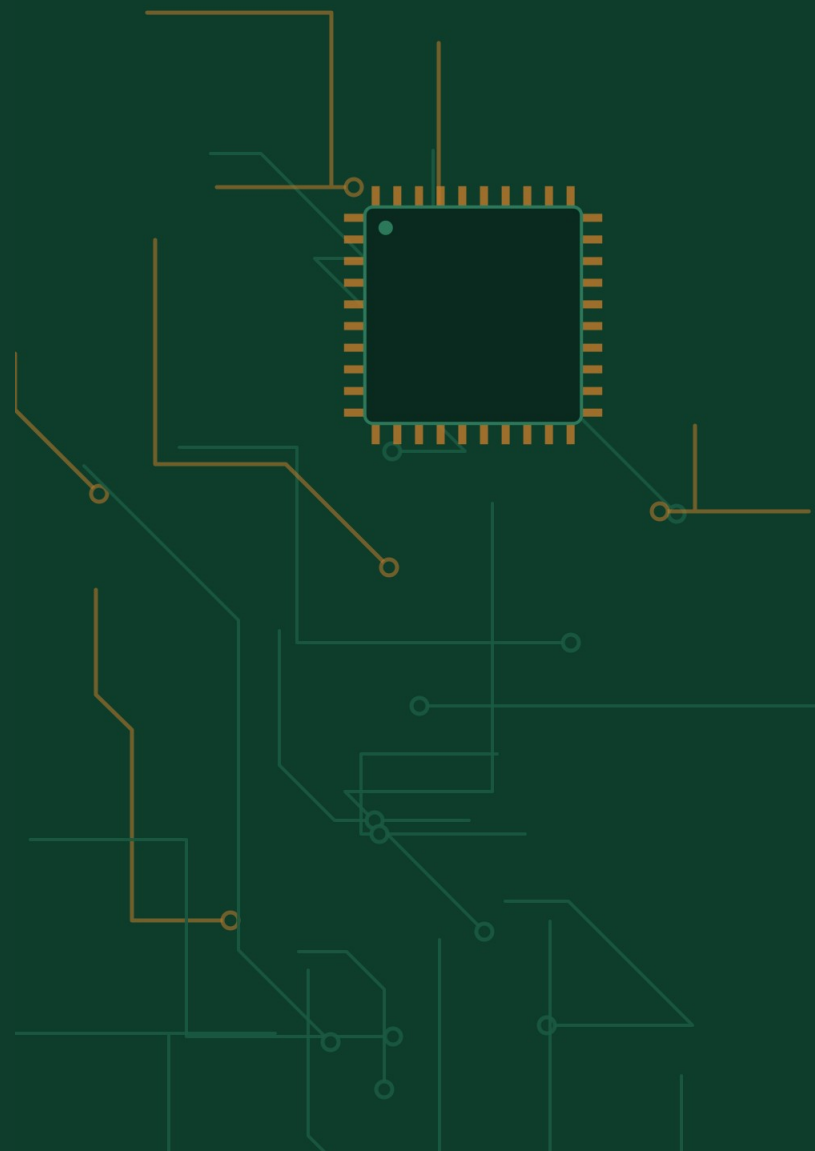
```
#if __has_include("rahasia.h")
#include "rahasia.h"            // board asli
#else
#define WIFI_SSID "Wokwi-GUEST"  // Wokwi
#define WIFI_SANDI ""
#define WIFI_KANAL 6
#endif
```

- `include/rahasia.h` berisi SSID & sandi asli
- Tercantum di `.gitignore` → tidak ikut di-*commit*
- Tanpa `rahasia.h` tetap lulus kompilasi (Wokwi-GUEST)
- Salin `rahasia.example.h` → `rahasia.h`, lalu isi

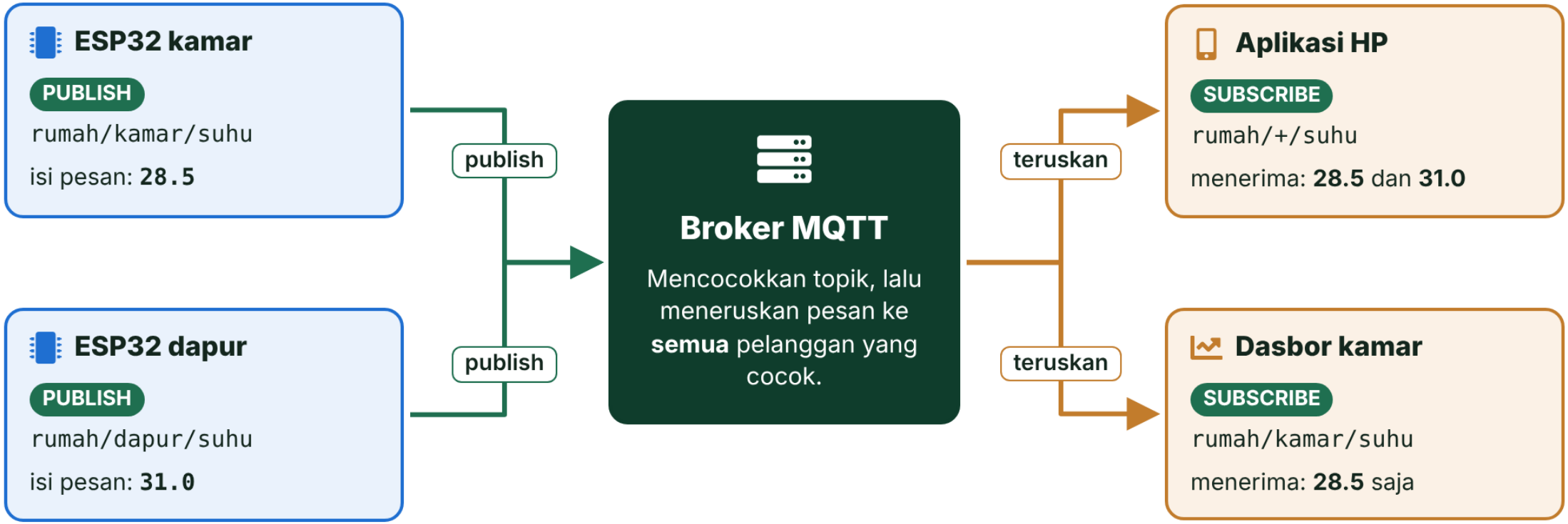
02

MQTT

Papan pengumuman untuk perangkat IoT



Pola Publish/Subscribe



Pengirim dan penerima tidak saling kenal dan tidak perlu tahu alamat IP satu sama lain. Mereka cukup sepakat soal **nama topik**.

Empat Unsur MQTT



Broker

Server yang menerima lalu meneruskan pesan, mis. broker.emqx.io.



Klien

ESP32, MQTTX, aplikasi HP. Bisa publish sekaligus subscribe.



Topik

Alamat bertingkat: `kuliah-st/<NIM>/sensor`.



Payload

Isi pesan: teks `ON` atau JSON.

Topik Bertingkat dan Wildcard

```
kuliah-st
├── 12345678
│   ├── sensor
│   ├── status
│   └── led
│       └── status
└── 87654321
    ├── sensor
    └── led
```

Filter langganan	Menerima pesan dari topik
`kuliah-st/12345678/sensor`	hanya topik itu (cocok persis)
`kuliah-st/12345678/+`	.../sensor, .../status, .../led tetapi bukan .../led/status
`kuliah-st/12345678/#`	semua topik milik 12345678, termasuk .../led/status
`kuliah-st/+/sensor`	data sensor **semua** mahasiswa

+ = tepat **satu** tingkat

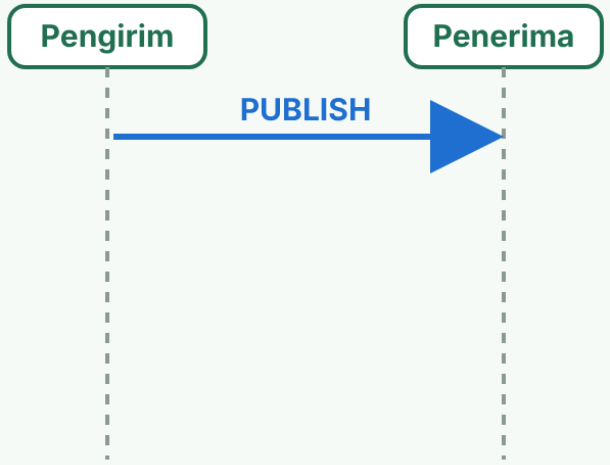
= semua tingkat di bawahnya, hanya di akhir

Peka huruf besar/kecil: `LED` ≠ `led`

Tiga Tingkat QoS

QoS 0

paling banyak sekali (*at most once*)



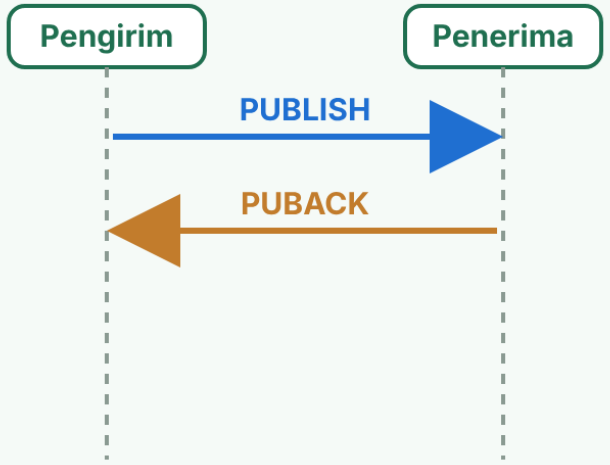
Tanpa balasan.

Bisa hilang, tidak pernah dobel.

Ibarat kartu pos.

QoS 1

minimal sekali (*at least once*)



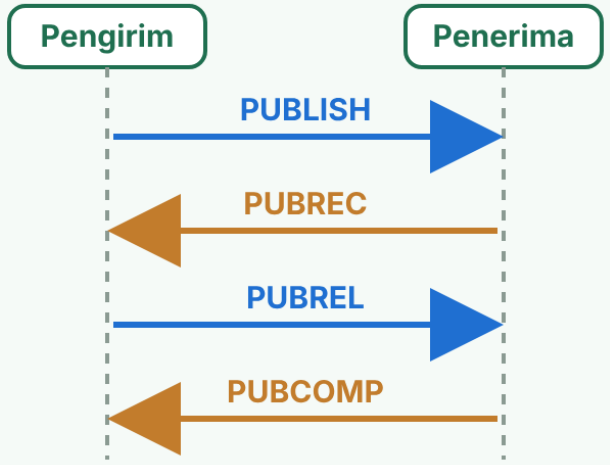
Dikirim ulang sampai ada PUBACK.

Tidak hilang, tetapi bisa dobel.

Ibarat paket dengan tanda terima.

QoS 2

tepat sekali (*exactly once*)



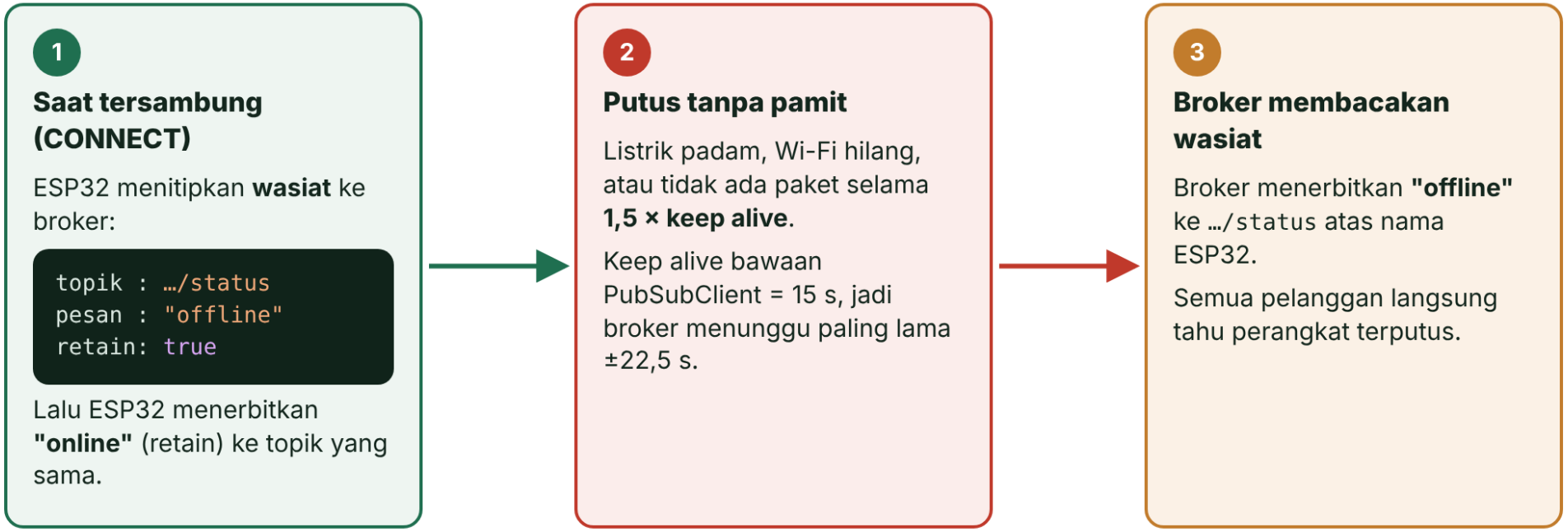
Empat paket: paling lambat.

Tidak hilang, tidak dobel.

Ibarat tanda terima dua tahap.

PubSubClient (pustaka yang kita pakai): **publish selalu QoS 0**, **subscribe boleh QoS 0 atau 1**. QoS yang sampai ke pelanggan = yang lebih kecil antara QoS pengirim dan QoS langganannya.

LWT dan Retained Message



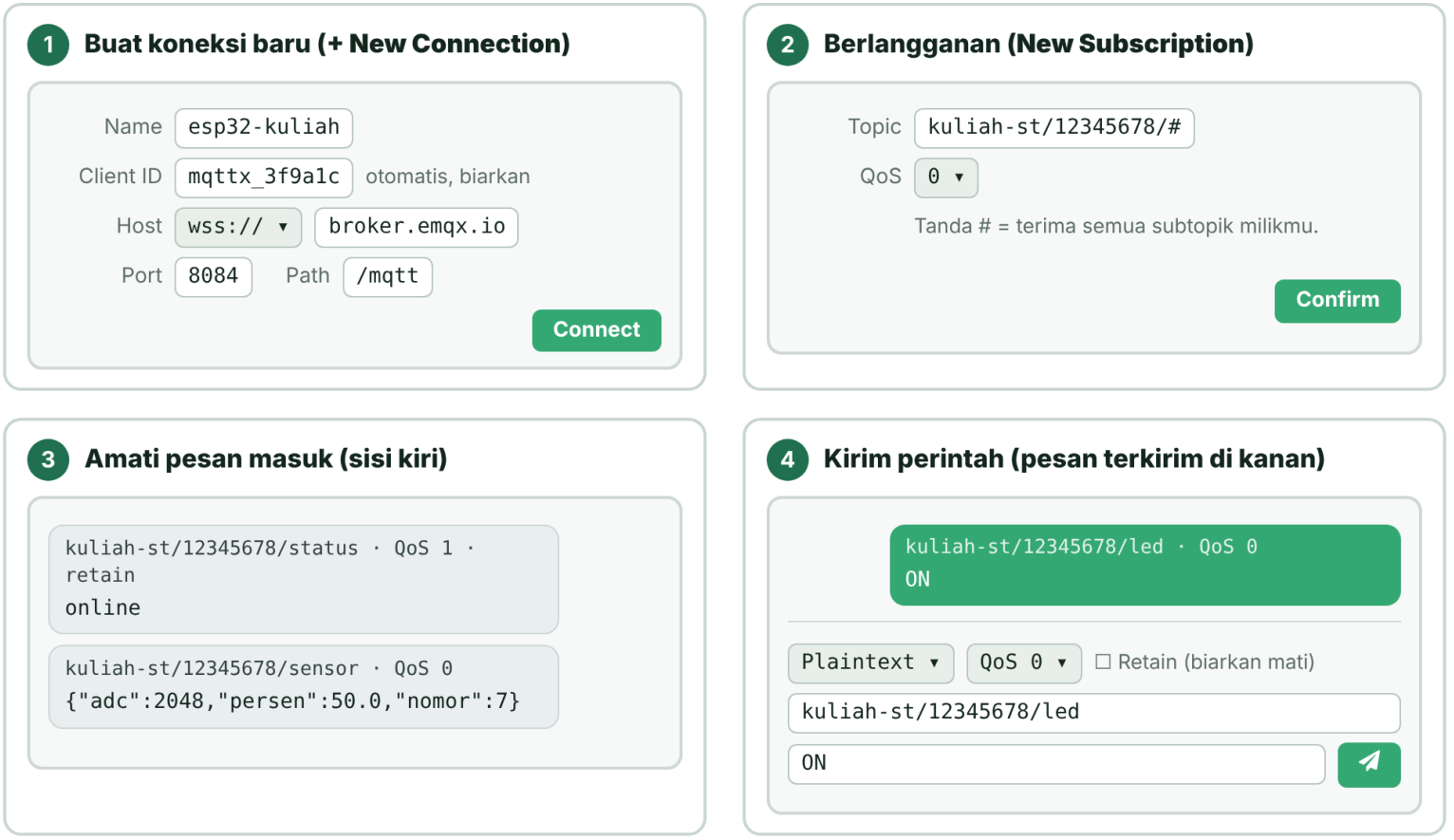
Retained message = pesan yang "ditempel" broker. Broker menyimpan **satu pesan terakhir** per topik dan langsung memberikannya kepada pelanggan yang baru datang. Karena itu, dasbor yang baru dibuka langsung melihat "online" atau "offline". Untuk menghapusnya, kirim pesan kosong dengan retain.

Jika ESP32 berpamitan dengan benar (paket DISCONNECT), wasiat dibuang dan tidak diterbitkan.

Bawaan PubSubClient 2.8

Pengaturan	Bawaan	Ubah dengan
Versi protokol	MQTT 3.1.1	—
Buffer paket	256 byte	setBufferSize()
Keep alive	15 detik	setKeepAlive()
Batas tunggu balasan	15 detik	setSocketTimeout()
QoS publish	0 saja	—
QoS subscribe	0 atau 1	subscribe(topik, 1)

MQTTX Web: Empat Langkah



Ilustrasi sederhana buatan sendiri, bukan tangkapan layar. Letak tombol di versi MQTTX-mu bisa sedikit berbeda.

Broker Publik = Ruang Publik



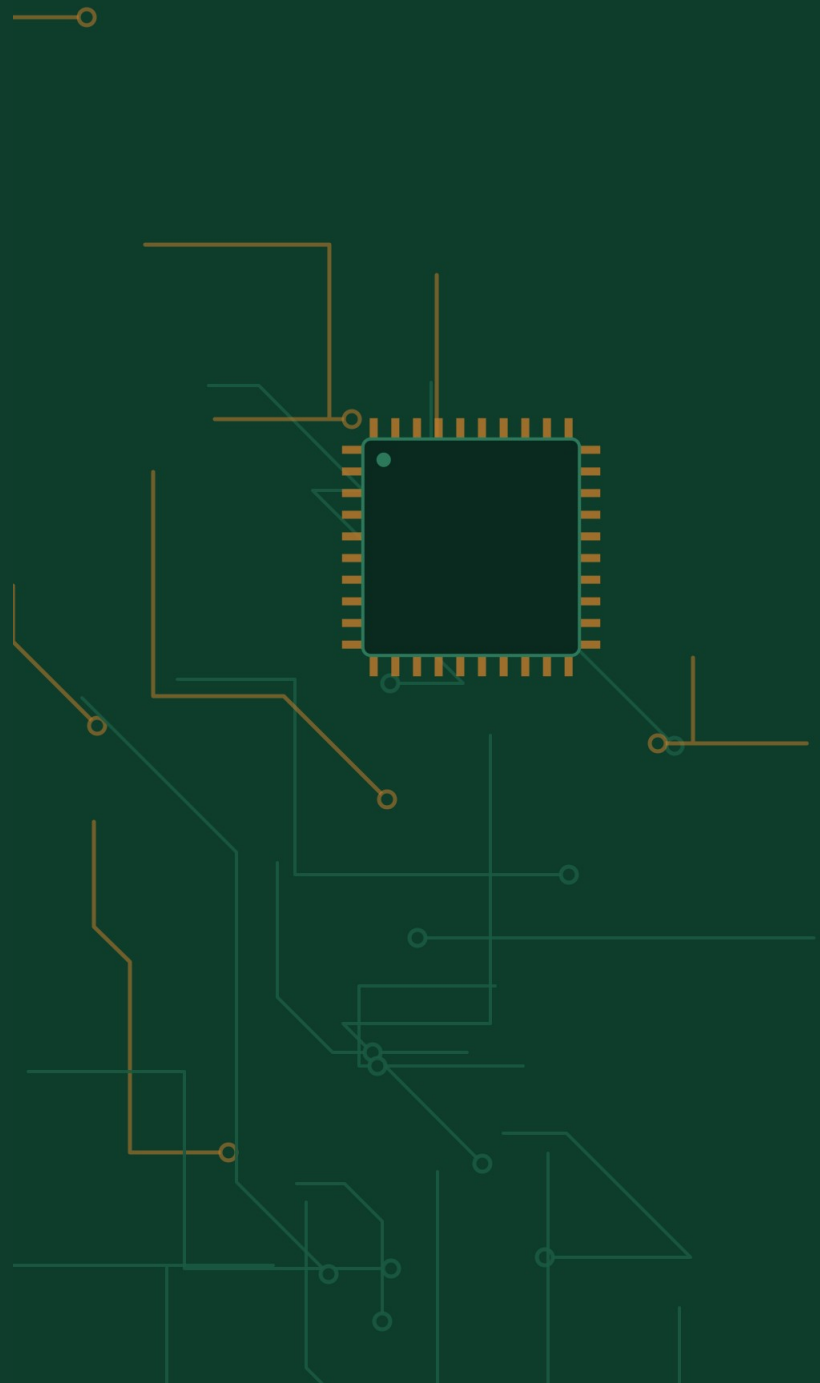
Siapa pun bisa **membaca dan menulis** topikmu. Pakai hanya untuk belajar.

- Jangan kirim data pribadi
- Jangan kendalikan alat berbahaya (pemanas, pompa, kunci pintu)
- Produk nyata: TLS (8883) + autentikasi + broker pribadi
- Sandi Wi-Fi tidak di-*commit* (`rahasia.h`)

03

Praktikum

Pindai Wi-Fi, kirim data, kendalikan LED



Praktikum 2: Kirim JSON tiap 5 Detik

kode/02-mqtt-kirim-data/src/main.cpp

```
char payload[80];
snprintf(payload, sizeof(payload),
  "{\"adc\":%d,\"persen\":%.1f,\"nomor\":%lu}",
  nilaiAdc, persen, nomorPesan);
bool ok = mqtt.publish(topikSensor, payload);
Serial.printf("[KIRIM] %s -> %s\n", payload,
              ok ? "OK" : "GAGAL");
```

- Potensiometer di **GPIO 34** (ADC1)
- Topik `kuliah-st/<NIM>/sensor`, QoS 0
- JSON memakai titik desimal: `50.0`
- Pantau di MQTTX: `kuliah-st/<NIM>/#`

Praktikum 3: Perintah ON/OFF

kode/03-mqtt-kendali-led/src/main.cpp

```
void pesanMasuk(char *topik, byte *isi,
                unsigned int panjang) {
  char perintah[16];
  unsigned int n = (panjang < 15) ? panjang : 15;
  memcpy(perintah, isi, n);
  perintah[n] = '\0';        // payload tanpa '\0'
  if (strcasecmp(perintah, "ON") == 0)
    ledMenyala = true;
  else if (strcasecmp(perintah, "OFF") == 0)
    ledMenyala = false;
  else return;               // abaikan perintah lain
  digitalWrite(PIN_LED, ledMenyala ? HIGH : LOW);
  perluLaporLed = true;      // publish dari loop()
}
```

- Subscribe `kuliah-st/<NIM>/led` (QoS 1)
- LWT `offline` + `online` (retain) di `…/status`
- Sambung ulang dijadwalkan `millis()`, tanpa `while`
- Jangan publish di dalam *callback*

Kuis Cepat

ESP32 berlangganan `kuliah-st/12345678/+`. Pesan dari topik mana yang TIDAK diterima?

A kuliah-st/12345678/led

B kuliah-st/12345678/status

C kuliah-st/12345678/led/status

D kuliah-st/12345678/sensor

Rangkuman

- IoT: perangkat → jaringan → **broker** → aplikasi
- ESP32 hanya **2,4 GHz**; STA = tamu, AP = tuan rumah; RSSI mendekati 0 = kuat
- MQTT: publish/subscribe lewat broker; wildcard `+` dan `#` hanya untuk subscribe
- QoS 0/1/2; retained = pesan terakhir; LWT = wasiat saat putus tanpa pamit
- Panggil `mqtt.loop()` sesering mungkin; sambung ulang pakai `millis()`
- Broker publik hanya untuk belajar; sandi di `rahasia.h`

Sebelum Pertemuan 14

Tugas

- Latihan 1: termometer daring DHT22
- Latihan 2: kendali dua arah dengan tombol

Persiapan

- Baca modul Pertemuan 14: Hemat Daya
- Ukur: berapa lama ESP32 hidup dengan baterai?

Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas